

Teaching a computer to simplify 3-manifold triangulations with reinforcement learning

Lian de Souza Li IMECC/UNICAMP, Brazil
Code: <https://github.com/USERNAME/pachner-rl>

October 2026

Abstract

A closed 3-manifold has infinitely many triangulations, and finding a small one is a search problem on the graph of Pachner moves. I train one shared reinforcement-learning (RL) agent, a Fitted-Q network that scores the legal 2-3, 3-2 and 4-4 moves, to shrink triangulations of Dehn fillings of cusped census manifolds. Dehn fillings are an endless source of closed instances with a computable reference size (the best of many randomized SnapPy simplifications; not a proven minimum). In a paired test in which every method starts from the same scrambled triangulation (742 non-trivial trials on five held-out fillings), the agent reaches the reference size in 56.5% of trials, against 44.2% for a greedy baseline and 52.2% for a greedy baseline that picks move types evenly: a clear gain over the first and a small one over the second. On scrambles of exactly 100 tetrahedra, far outside its training range, the agent does not beat greedy search. All results are produced by the accompanying repository.

1 Introduction and related work

A closed 3-manifold M can be assembled by gluing tetrahedra along their faces. The same manifold has infinitely many such *triangulations*, of very different sizes, and a smaller one makes almost every algorithm faster, from 3-sphere recognition [2] to invariant computation (software such as Regina [5] and SnapPy [6] simplify triangulations routinely). Pachner’s theorem [1] says that any two triangulations of M are joined by a finite sequence of local *Pachner moves*, but not how long the sequence must be, nor how it should be chosen. Greedy descent, which only applies size-reducing moves, gets stuck in local minima, and escaping them may require *growing* the triangulation first. For 3-sphere triangulations the necessary growth seems small: Burton’s experiments on about 3×10^7 one-vertex triangulations with up to nine tetrahedra never needed more than two extra tetrahedra [3]; no such bound is known in general.

Machine learning has recently been applied to closely related combinatorial searches in low-dimensional topology. Gukov et al. [7] used RL to find Reidemeister-move sequences that unknot knot diagrams; Applebaum et al. [8] used RL on diagrams with up to 200 crossings and determined the unknotting number of tens of thousands of knots; Sheper et al. [9] studied why rare, high-reward instances make Andrews–Curtis presentation problems hard for RL, and Costantino, He, Heyes and Hirst [10] analysed Pachner graphs of 3-manifold triangulations with network analysis and machine learning. Learned heuristics for large move graphs also solve puzzles such as the Rubik’s cube [11]. I ask the analogous question for triangulation simplification: *can a single agent, trained across many manifolds, learn a useful policy for choosing Pachner moves?* The answer found here is a measured “partly”: a small advantage over strong hand-written heuristics on in-range instances, and none far outside the training range.

2 Background

2.1 Triangulations and Pachner moves

I work with one-vertex triangulations of closed orientable 3-manifolds, represented by SnapPy’s `t3mlite Mcomplex`. Three Pachner moves are used (Figure 1): the **2-3** move replaces two tetrahedra sharing a face by three tetrahedra around a new edge (+1 tetrahedron, available across any interior face); its inverse, the **3-2** move (−1), needs an edge of valence exactly 3; and the **4-4** move (± 0) re-triangulates the four tetrahedra around a valence-4 edge. The 1-4 and 4-1 moves are excluded because they change the number of vertices; for one-vertex triangulations the 2-3 and 3-2 moves alone already connect all triangulations of a manifold (Matveev, Piergallini; see [4]). The *size* n of a triangulation is its number of tetrahedra, and the task is to reach a small n using the fewest, cheapest moves.

Why greedy fails. Applying 3-2 moves while one exists always lowers n , but it stops when no valence-3 edge remains. 2-3 moves create valence-3 edges (the new edge) and 4-4 moves reshuffle valences at no cost, so good paths mix all three moves; which mix is the learning problem.

2.2 Dehn fillings

Let M be a cusped manifold such as a knot complement, with torus boundary. Gluing a solid torus so that its meridian disc is attached along the slope $p\mu + q\lambda$ gives a closed manifold $M(p, q)$. SnapPy’s `filled_triangulation()` returns a closed one-vertex triangulation of $M(p, q)$; I use these as start states. For example the figure-eight knot complement 4_1 with slope $(5, 1)$ gives a closed manifold with $H_1 = \mathbb{Z}/5$ whose best-known triangulation has nine tetrahedra.

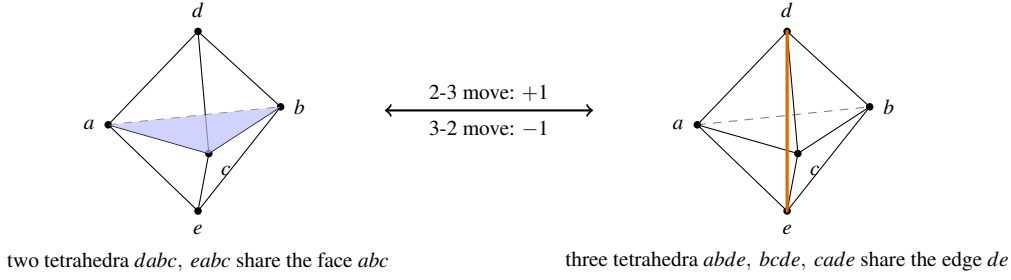


Figure 1: The 2-3 / 3-2 move. The five vertices bound a bipyramid, split into two tetrahedra by the shared face abc (blue) or into three by the new edge de (orange, valence 3).

2.3 Simplification as a Markov decision process

State. Six hand-made features of the current triangulation: n/n_0 , steps left over the budget, the fractions of edges with valence ≤ 3 , $= 4$ and ≥ 6 , and the edge count over n_0 (n_0 is the starting size). **Actions.** Every legal 2-3, 3-2 and 4-4 move, each described by six features (type one-hot, change in n over 3, mean and maximum valence of the edges involved). **Reward.** For a successful move, $r = (n_t - n_{t+1}) - 0.02$; a failed move costs -1 ; reaching the target size gives $+5$ and ends the episode (the agent is not told the target in its state). **Learner.** A Fitted-Q agent [12]: an MLP (two hidden layers of 64 ReLU units, `scikit-learn`) maps (state, action) features to Q and is regressed by Adam steps on minibatches from a replay buffer toward $r + \gamma \max_{a'} Q(s', a')$ with $\gamma = 0.97$. Because the network scores each legal move, a single network handles triangulations of any size and any number of legal moves. Exploration is ϵ -greedy with *type-balanced* random moves (a move type is drawn first, then a move of that type), since a uniform draw over the move list would almost always pick a 2-3 move. **Training loop.** At every step the agent picks a move (ϵ -greedy), the environment applies it, the transition (features, reward, next legal moves) goes to a replay buffer (40 000 transitions), and one minibatch of 256 transitions is regressed on the Fitted-Q targets. An episode ends at the target, at a step budget (≤ 50 in training) or if $n > 3n_0 + 10$. Two details matter. First, the agent receives only generic features, never the target or the manifold, so one network serves every filling. Second, the next-state maximum ranges over *all* legal moves, so the learned Q can value a 2-3 move (a temporary -1 in reward) when it opens later 3-2 moves.

3 Data

Closed instances from Dehn fillings. Cusped census manifolds and coprime slopes (p, q) give an unlimited supply of closed manifolds. Reference sizes are needed to know when a run succeeds, so for each filling I run 200 randomized SnapPy simplifications (`randomize()` followed by `simplify()`) on the filled manifold and keep the smallest triangulation found: the *best-known* size, an upper bound for the true minimum. Over the 11 training and 5 held-out fillings, SnapPy’s own start triangulation is 0–2 tetrahedra above the best-known size, so the room to improve is small, which makes every tetrahedron count. SnapPy’s filled triangulations are random, so each filling’s start triangulations are drawn once (a pool of up to 20) and stored by isomorphism signature in the repository; all evaluations reuse them.

Training pool (11 fillings from six sources): $4_1(5, 1)$, $4_1(8, 1)$, $4_1(9, 1)$, $5_2(9, 1)$, $5_2(11, 1)$, $6_1(5, 1)$, $6_1(9, 1)$, $6_2(7, 1)$, $6_3(5, 1)$, $m006(5, 1)$, $m006(7, 1)$. **Held-out test fillings** (unseen (knot, slope) pairs):

filling	SnapPy identification	H_1	best-known n	start sizes in pool
$4_1(7, 2)$	–	$\mathbb{Z}/7$	10	11–12
$5_2(7, 1)$	m015(5, 1)	$\mathbb{Z}/7$	10	10–11
$6_1(7, 1)$	–	$\mathbb{Z}/7$	16	16–17
$6_3(7, 1)$	s912(7, 1)	$\mathbb{Z}/7$	17	17–19
m009(5, 1)	m009(5, 1)	$\mathbb{Z}/2 + \mathbb{Z}/4$	11	11–11

Fillings such as $5_2(7, 1)$ and $6_3(7, 1)$ are filled census manifolds (m015, s912) that also appear as training sources, so “held-out” means an unseen slope or knot, not an unrelated manifold family.

What the data look like. SnapPy’s filled triangulations are already within one tetrahedron of the best-known size for 94% of the pool (Figure 2, left), so on its own a filled triangulation is a nearly trivial instance. The scrambles create the difficulty: after $K = 5, 10, 20$ extra moves the median test start is 3, 5 and 9 tetrahedra above target, and the agent must undo that growth. Because the true minimum is unknown for real fillings, a success only means “matched the best SnapPy found”; a run that beat it would count as a success too, and none did.

Scrambles. An episode starts from a pooled triangulation after K extra growth-biased random moves (shrink bias -0.6): $K \in \{5, 10, 20\}$ at test time, 0–10 during training. A start that is already at or below the target is counted as trivial and excluded from success rates. To teach the agent the basic idea before the harder real data, training also uses **synthetic** episodes: census manifolds with known minimal size (m003, m004, m015, m032, s912; $n = 2$ –6) scrambled by 2–30 moves, with the exact minimum as target. **Curriculum** (900 episodes, one shared agent): the synthetic share falls from 70% to 30%, scramble depth grows, and ϵ falls from 0.9 to 0.05. The agent sees neither the target nor whether an episode is real or synthetic.

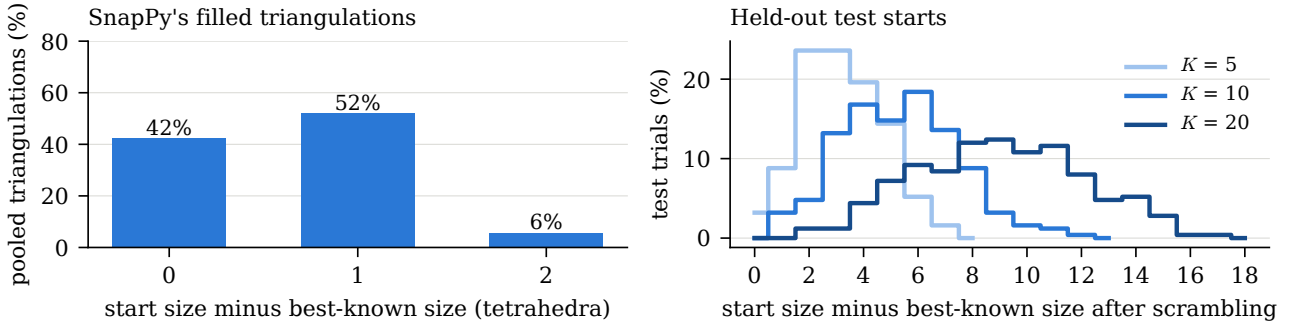


Figure 2: Left: how far SnapPy’s own filled triangulations are above the best-known size (all 16 fillings, pooled). **Right:** the same excess for the test starts after K growth-biased scramble moves.

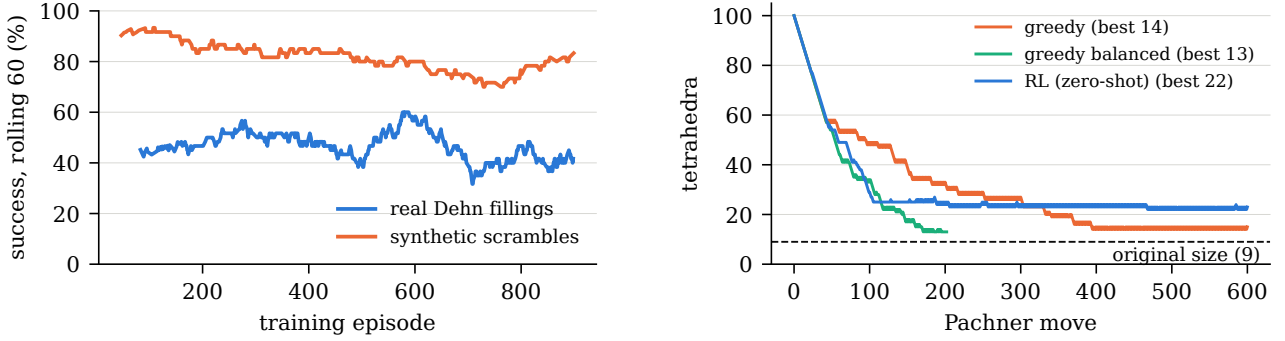


Figure 3: Left: training success (rolling mean over 60 episodes of each kind). **Right:** one 100-tetrahedron scramble of $4_1(5, 1)$: identical start, 600-move budget; “best” is the smallest size reached.

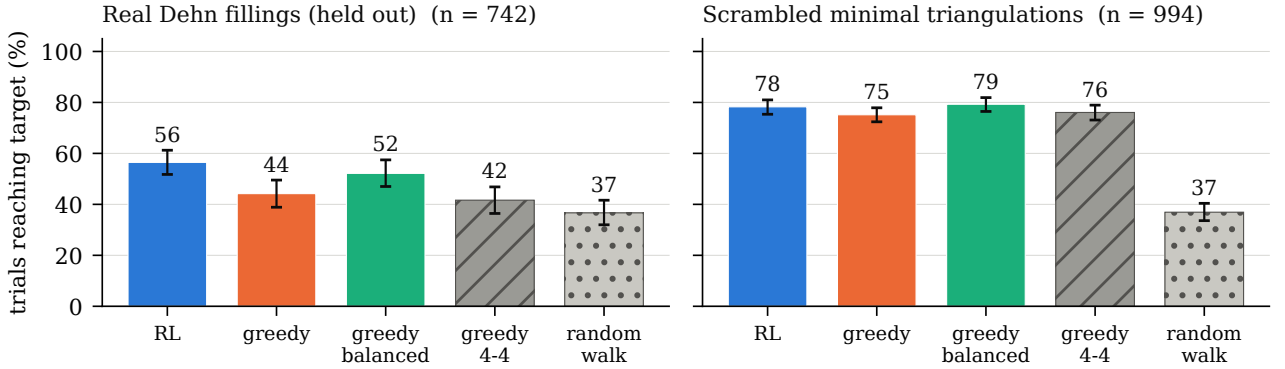


Figure 4: Share of non-trivial trials reaching the target (95% cluster-bootstrap intervals). Left: five held-out fillings, $K \in \{5, 10, 20\}$, 50 seeds. Right: scrambles of five minimal census manifolds, depths 5–30, 50 seeds.

4 Results

4.1 Training

Over the last 300 of 900 episodes the agent solved 78% of synthetic episodes ($n = 97$) and 42% of real ones ($n = 203$; 38% excluding trivial starts); the largest training start had 27 tetrahedra (Figure 3, left). The synthetic curve declines as scrambles deepen and the real curve is roughly flat while its difficulty grows. These are training-pool numbers and say nothing about generalization.

4.2 Held-out comparison with paired trials

Every method starts from an *identical* scrambled triangulation (same pooled start, same seeded scramble), so differences cannot come from easier starts; an earlier unpaired test with 10 trials per method had almost no power. Success means reaching the best-known size (real) or the known minimum (synthetic). There are 50 seeds per (filling, K) and per (manifold, depth); uncertainty is a 95% bootstrap interval over (filling, seed) clusters, and sign tests use trials where exactly one of two methods succeeds. Compared methods: the RL agent ($\epsilon = 0.05$); *greedy*, a 3-2 move if one exists and otherwise a random move from the whole list; *balanced greedy*, the same with a type-balanced random fallback; greedy with a 4-4 fallback; and a biased random walk.

	success, % [95% CI]	RL minus method, points [95% CI]	RL-only / other-only
<i>Real held-out fillings</i> (742 non-trivial trials)			
RL agent	56.5 [51.7, 61.2]	—	
greedy (flat fallback)	44.2 [38.9, 49.5]	+12.3 [+8.5, +16.0]	139/48
greedy (balanced fallback)	52.2 [47.0, 57.4]	+4.3 [+0.8, +8.1]	100/68
greedy (4-4 fallback)	41.6 [36.4, 46.8]	+14.8 [+11.2, +18.5]	143/33
biased random walk	36.7 [32.0, 41.6]	+19.8 [+15.3, +24.2]	198/51
<i>Scrambled minimal manifolds</i> (994 non-trivial trials)			
RL agent	78.3 [75.4, 81.0]	—	
greedy (flat fallback)	75.2 [72.4, 77.9]	+3.1 [+0.1, +6.0]	133/102
greedy (balanced fallback)	79.3 [76.5, 81.9]	−1.0 [−4.1, +2.0]	96/106
greedy (4-4 fallback)	76.1 [73.1, 78.9]	+2.2 [−0.9, +5.3]	118/96
biased random walk	36.9 [33.6, 40.4]	+41.3 [+37.7, +44.9]	437/26

Table 1: Paired comparison. “RL-only / other-only” counts trials in which exactly one of the two methods reached the target.

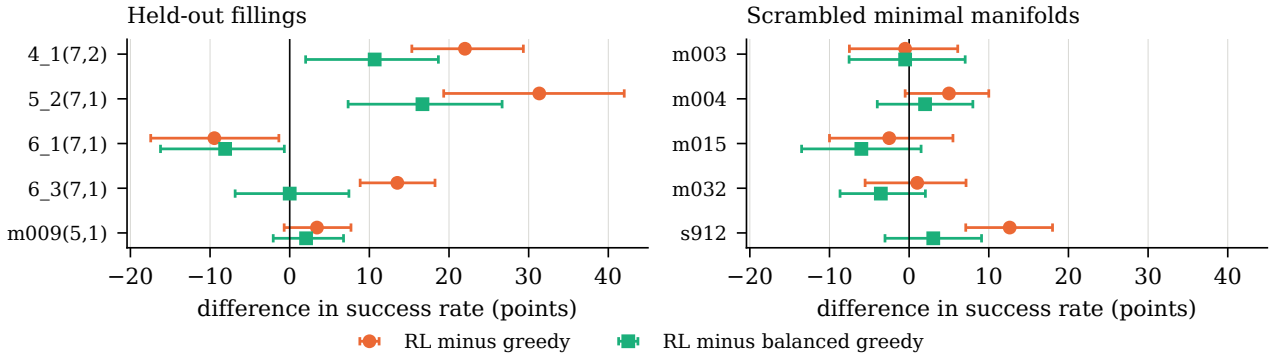


Figure 5: Difference in success rate, RL minus each greedy baseline, per instance (95% bootstrap intervals over seeds). Positive values favour RL.

setting	n	RL	greedy	balanced	4-4 greedy	random walk
fillings, K 5	242	53	43	51	44	41
fillings, K 10	250	54	43	54	41	34
fillings, K 20	250	62	47	52	40	35
minimal, depth 5	245	92	93	95	95	64
minimal, depth 10	249	88	87	89	88	50
minimal, depth 20	250	75	72	76	70	26
minimal, depth 30	250	58	48	57	52	8

Table 2: Success (%) by scramble length, non-trivial trials.

Aggregate. On the real held-out fillings the agent reaches the target in 56.5% of trials. This is clearly above the first greedy baseline (+12.3 points, CI [+8.5, +16.0]) and above the 4-4 and random-walk baselines, but only modestly above the balanced greedy (+4.3 points [+0.8, +8.1], sign test $p = 0.017$, not corrected for four comparisons). An earlier run with freshly drawn start triangulations gave +1.2 [−2.1, +4.6] against the balanced greedy, so I trust the sign of this gap but not its size. On synthetic scrambles the agent and balanced greedy are tied (−1.0 points [−4.1, +2.0]), and all three greedy variants beat the random walk by roughly 40 points. The policies differ: by attempted moves the agent plays 45% 3-2, 36% 2-3 and 18% 4-4 on real fillings, against 51/44/5 for flat greedy and 42/32/25 for balanced greedy. In none of the 750 real and 1000 synthetic trials did any method finish strictly below the target.

Where the agent helps. The aggregate hides large differences (Figure 5). Against balanced greedy the agent is ahead on $5_2(7,1)$ and $4_1(7,2)$, level on $6_3(7,1)$ and $m009(5,1)$, and *behind* on $6_1(7,1)$; on synthetic scrambles it is level with balanced greedy on every manifold, and ahead of the first greedy only on s912. I do not know why $6_1(7,1)$ is the exception; a plausible but untested explanation is that its start triangulations need a kind of 4-4 rearrangement that the six features cannot distinguish. Table 2 shows that the agent’s lead on real fillings is largest for the longest scrambles ($K = 20$: 62% against 47% and 52%), while on synthetic scrambles all heuristics lose ground together as the depth grows and the agent’s lead over balanced greedy is nil.

4.3 Scaling to 100 tetrahedra

To test scaling, I grow the base triangulation of $4_1(5,1)$ to exactly 100 tetrahedra with random 2-3 (and some 4-4) moves; the target is the original size, 9 tetrahedra, with budgets of 180 or 600 moves. Every final triangulation that I checked (the first three scrambles of each method and configuration; 90 runs) still has the original homology and volume. Applied unchanged,

	$4_1(5, 1)$, target 9		$5_2(7, 1)$, target 10	
move budget	180	600	180	600
RL zero-shot ($\epsilon=0.05$)	0 (24.2)	2 (14.5)	0 (25.7)	2 (19.0)
RL fine-tuned ($\epsilon=0.05$)	1 (19.3)	3 (16.6)	1 (23.3)	2 (19.3)
RL zero-shot ($\epsilon=0$)	1 (25.3)	0 (19.9)	0 (27.9)	1 (23.5)
RL fine-tuned ($\epsilon=0$)	1 (20.5)	2 (15.8)	1 (23.4)	1 (22.9)
greedy	0 (24.5)	8 (11.3)	0 (28.7)	5 (14.2)
greedy balanced	3 (19.5)	5 (15.8)	1 (22.6)	7 (17.2)

Table 3: 100-tetrahedron scrambles, 15 per cell: number returned to the original size (mean smallest size reached in brackets). ϵ is the exploration rate at test time.

the agent—trained on starts of at most 27 tetrahedra—returns to the original size in 0/20 scrambles with 180 moves and 4/20 with 600, against 1/20 and 11/20 for greedy and 1/20 and 4/20 for balanced greedy. All methods fall quickly from 100 to about 25 tetrahedra (Figure 3, right); the question is how often they get all the way back. The agent also costs more per run (1.1–2.1 s against 0.3–0.6 s for greedy), as it scores every legal move with the network.

I then fine-tuned a copy of the agent for 200 episodes on scrambles growing from 20 to 100 tetrahedra, with $\gamma = 0.99$, a frozen target network re-synced every 150 updates and clipped targets (a first attempt with plain bootstrapping diverged), and compared all methods on fresh scrambles of $4_1(5, 1)$ and of an unseen manifold, $5_2(7, 1)$ (Table 3). During the last 50 fine-tuning episodes (exploration on) the agent returned to the original size from 7 of 41 starts with at least 90 tetrahedra, so it learned something on the training scrambles.

Fine-tuning helps at the short budget (mean smallest size $4_1(5, 1)$: $24.2 \rightarrow 19.3$), where the fine-tuned agent matches balanced greedy, but it does not help at 600 moves, where plain greedy is best on both manifolds (8/15 and 5/15 returns, against 3/15 and 2/15 for the fine-tuned agent). So the answer to “does training at the target scale fix the scaling problem?” is no, at least with 200 episodes and six-feature states. A plausible reason greedy does well here is that the scrambles are built from 2-3 moves, which 3-2 moves simply reverse; I did not test this. With $\epsilon = 0$ the agent can cycle between a 2-3 move and its inverse, which is why $\epsilon = 0.05$ is used throughout.

5 Conclusion and extensions

What the evidence says. One shared agent, trained for 900 episodes on triangulations of at most 27 tetrahedra, chooses Pachner moves better than a naive greedy search and slightly better than a type-balanced greedy search on held-out fillings, with a filling-dependent margin. It does not reach 100 tetrahedra, even after fine-tuning. The main methodological lesson is that a baseline which merely balances move types removes most of the apparent RL gain, so paired comparisons against strong, cheap baselines are essential before claiming that RL has learned something.

Reflections. The evaluation design mattered more than the learning algorithm: the numbers that decide the story (a clear gain over one greedy rule, a small one over another, a loss at scale) only appeared once every method started from the same triangulation, once baselines were given a fair chance, and once the test was moved to unseen fillings and to a much larger size. Dehn fillings were a convenient source of closed instances, but SnapPy’s start triangulations are already close to the best-known size, so the headroom (0–2 tetrahedra) is small, which caps what any method can show and makes differences of a few points fragile. A learned policy is also several times more expensive per run than a hand-written rule ($3\text{--}4\times$ at 100 tetrahedra), so it must clearly win to be worth using; at 100 tetrahedra it does not.

Limits. Reference sizes are best-known upper bounds, not proven minima, and the room to improve is only 0–2 tetrahedra. There is one training run, so seed-to-seed variance of the agent is unmeasured; five held-out fillings, which share families with the training sources, give a narrow test; and six hand-made features discard most of the triangulation.

Extensions. (i) A graph neural network over the tetrahedron gluing graph would see the real combinatorics and could scale in size. (ii) More diverse training manifolds (larger fillings, other census families) and several training seeds would test generalization properly. (iii) Instances where growth is provably necessary, such as the sphere triangulations in Burton’s study, would show whether RL finds genuinely non-greedy escapes rather than a better-tuned greedy rule.

Reproducibility and use of LLMs. Code, pinned environment, the trained checkpoint, the stored start triangulations and instructions are at <https://github.com/USERNAME/pachner-rl>; `scripts/reproduce.sh` quick takes about two minutes and `full` regenerates every table and figure here. Claude (Anthropic) assisted with code, analysis and drafting; I ran and checked the code, and every number above is generated by the repository scripts from stored results.

References

- [1] U. Pachner, P.L. homeomorphic manifolds are equivalent by elementary shellings, *European J. Combin.* 12 (1991) 129–145.
- [2] A. Thompson, Thin position and the recognition problem for S^3 , *Math. Res. Lett.* 1 (1994) 613–630.
- [3] B. A. Burton, The Pachner graph and the simplification of 3-sphere triangulations, *Proc. SoCG* 2011, 153–162 (arXiv:1011.4169).
- [4] S. Matveev, *Algorithmic Topology and Classification of 3-Manifolds*, 2nd ed., Springer, 2007.
- [5] B. A. Burton, R. Budney, W. Pettersson et al., Regina: software for low-dimensional topology, <https://regina-normal.github.io>.
- [6] M. Culler, N. M. Dunfield, M. Goerner, J. R. Weeks, SnapPy, a computer program for studying the geometry and topology of 3-manifolds, <https://snappy.computop.org>.
- [7] S. Gukov, J. Halverson, F. Ruehle, P. Sułkowski, Learning to unknot, *Mach. Learn.: Sci. Technol.* 2 (2021) 025035.
- [8] T. Applebaum, S. Blackwell, A. Davies, T. Edlich, A. Juhász, M. Lackenby, N. Tomašev, D. Zheng, The unknotting number, hard unknot diagrams, and reinforcement learning, arXiv:2409.09032 (2024).
- [9] A. Shehper et al., What makes math problems hard for reinforcement learning: a case study, arXiv:2408.15332 (2024).
- [10] F. Costantino, Y.-H. He, E. Heyes, E. Hirst, Learning 3-manifold triangulations, *J. Phys. A* 58 (2025) 095201 (arXiv:2405.09610).
- [11] F. Agostinelli, S. McAleer, A. Shmakov, P. Baldi, Solving the Rubik’s cube with deep reinforcement learning and search, *Nat. Mach. Intell.* 1 (2019) 356–363.
- [12] M. Riedmiller, Neural fitted Q iteration, *ECML 2005*, LNCS 3720, 317–328.

A Details

Hyperparameters. MLP ($12 \rightarrow 64 \rightarrow 64 \rightarrow 1$), ReLU, Adam (lr 10^{-3}), $\gamma = 0.97$ (0.99 for 100-tet fine-tuning), minibatch 256, replay 40 000, step cost 0.02, failed move -1 , success bonus $+5$, early stop at $n > 3n_0 + 10$, training step budget $\min(50, 3 \cdot \text{extra} + 20)$ (real) or $\min(45, 3 \cdot \text{depth} + 10)$ (synthetic); exploration ε decays linearly from 0.9 to 0.05 over the first 70% of the 900 episodes. 100-tet fine-tuning: scramble sizes 20–30 up to 100 over the first 60% of episodes, budgets 180/600, target network re-synced every 150 updates, targets clipped to $[-10, 100]$.

Per-filling success (%) on the real held-out fillings, non-trivial trials ($n = \text{trials}$):

filling	n	RL	greedy	balanced	4-4 greedy	random walk
$4_1(7, 2)$	150	35	13	25	13	17
$5_2(7, 1)$	150	44	13	27	11	12
$6_1(7, 1)$	148	70	80	78	75	61
$6_3(7, 1)$	148	36	23	36	22	20
m009(5, 1)	146	97	94	95	88	74